Day 3 – DRF Basics: Game & Publisher APIs

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Virtual Environment active**: Ensure your terminal shows `(.venv)` in the prompt.
   - **Django Server running**: Run `python manage.py runserver` to ensure the project boots up.
   - **Browser window ready**: Have your browser open to `http://127.0.0.1:8000/api/`.
2. **Prerequisites**:
   - You must have completed Day 2 successfully (models created, database migrated, and superuser credentials configured).

🎯 Learning Objectives

By the end of this class, you will be able to:

- Explain REST architecture basics and HTTP methods.
- Define DRF Serializers to handle translation between JSON and Django database models.
- Implement ModelSerializers and control data visibility (e.g., hiding secrets).
- Create ModelViewSets to handle CRUD routes automatically.
- Register endpoints with DRF's `DefaultRouter` and interact with the browsable API.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus
1. Hook & Big Picture	15 min	Define RESTful services. Compare standard Django views with Django REST Framework (DRF).
2. Key Concepts & Core Ideas	15 min	Explain Serializers as translators, ViewSets vs APIViews, and routers.
3. Live Coding Walkthrough	45 min	Configure settings, write serializers and viewsets, wire URLs, and test the endpoints.

4. Check for Understanding	10 min	Q&A on serialization validation, viewset mappings, and security exclusions.
5. Class Wrap-up & Checkpoint	5 min	Verify GET and POST requests are functional in the browsable API UI.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (15 min)

How does a mobile app or a separate frontend website talk to our Django database?

- **Q&A**:
  - **Question**: How does a mobile app or a separate frontend website talk to our Django database?
    - *Answer*: They communicate via HTTP REST APIs. Django REST Framework serializes database objects to JSON, which standard frontend apps can easily consume.
- **REST & DRF Overview**: Show how DRF streamlines this process:
  - **Serializers**: Handle validation and translation of Python models to and from JSON.
  - **ViewSets**: Standardize operations on resources (create, list, retrieve, update, delete).
  - **Routers**: Generate consistent, predictable URLs automatically.

2. Key Concepts & Core Ideas (15 min)

Introduce the core mechanics of DRF:

- **Serializer = Translator**: Converts complex Django model querysets/instances into Python datatypes that can be rendered to JSON (serialization) and parses incoming JSON payload data into validated Python dictionaries (deserialization).
- **`ModelSerializer`**: A shortcut class that auto-generates serializer fields matching the model fields.
- **ViewSet vs APIView**:
  - `APIView` requires manual handling of HTTP methods (e.g. implementing `get()` or `post()`).
  - `ViewSet` groups standard CRUD operations (`list`, `create`, `retrieve`, `update`, `destroy`) into a single class.
- **`DefaultRouter`**: Automatically generates clean URL patterns for all operations registered on a ViewSet.
- **Browsable API**: A built-in DRF feature that provides a web-based client interface for testing API endpoints during development.

HTTP Method	URL	ViewSet Action	CRUD Operation	SQL equivalent

GET	/api/games/	list	Read (List)	SELECT *
GET	/api/games/1/	retrieve	Read (Detail)	SELECT ... WHERE id=1
POST	/api/games/	create	Create	INSERT INTO ...
PUT	/api/games/1/	update	Update (Full)	UPDATE ... (all fields)
PATCH	/api/games/1/	partial_update	Update (Partial)	UPDATE ... (some fields)
DELETE	/api/games/1/	destroy	Delete	DELETE FROM ...

3. Live Coding Walkthrough (45 min)

Step 3.1: Enable Rest Framework

- **Concept**: Register the DRF app and our custom `games` app in Django settings so Django loads them.
- **Code**: Add to `INSTALLED_APPS` in `gamekey_platform/settings.py`:

```
INSTALLED_APPS = [
    ...
    'rest_framework', # Enable Django REST Framework
    'games',          # Register our custom app
]
```

Step 3.2: Writing Serializers

- **Concept**: Create `games/serializers.py` to translate our models. We exclude `webhook_secret` in `PublisherSerializer`—never return webhook secret keys in public API responses.
- **Code**: Create `games/serializers.py`:

```
from rest_framework import serializers
from .models import Game, Publisher


class GameSerializer(serializers.ModelSerializer):
    class Meta:
        model = Game
        # '__all__' is a shortcut to include all model columns (id, title, publisher, price)
        fields = '__all__'


class PublisherSerializer(serializers.ModelSerializer):
    class Meta:
        model = Publisher
        # exclude takes a list of fields that must NOT be exposed in the serialized API resp
```

```
        exclude = ('webhook_secret',)  # never expose the secret key to clients!
```

- **Verify**: Run `python manage.py check` to make sure the serializers do not have syntax or naming mistakes.
- **Q&A**:
  - **Question**: Why do we exclude `webhook_secret` from the publisher serializer?
    - *Answer*: Hiding the secret from GET/POST responses ensures that only our backend and the publisher know it, preventing malicious actors from stealing the secret to forge requests.
- ⚠ **Common Pitfalls**:
  - **Exposing sensitive fields**: Note that using `fields = '__all__'` on models containing secret credentials leaks data. Always use `exclude` or explicitly list fields when dealing with secrets.

Step 3.3: Creating ViewSets

- **Concept**: Create `games/viewsets.py` to define the database queries and serializers that our endpoints will use.
- **Code**: Create `games/viewsets.py`:

```
from rest_framework import viewsets
from .models import Game, Publisher
from .serializers import GameSerializer, PublisherSerializer


class GameViewSet(viewsets.ModelViewSet):
    # Define the query set that retrieves all records from the database
    queryset = Game.objects.all()
    # Define the translator serializer class to convert the objects
    serializer_class = GameSerializer


class PublisherViewSet(viewsets.ModelViewSet):
    queryset = Publisher.objects.all()
    serializer_class = PublisherSerializer
```

- **Verify**: Ensure the queryset and serializer classes are imported correctly.
- ⚠ **Common Pitfalls**:
  - **Missing queryset or serializer**: If either is missing on `ModelViewSet`, Django will throw an `AssertionError` during startup.

Step 3.4: Wiring URLs via Router

- **Concept**: Initialize DRF's `DefaultRouter`, register the viewsets, and include the generated routes in the primary Django URL configurations.
- **Code**: Modify `gamekey_platform/urls.py`:

```
from django.contrib import admin
```

```
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from games.viewsets import GameViewSet, PublisherViewSet

# Initialize the router
router = DefaultRouter()
# Register the viewsets: the first argument is the prefix, the second is the ViewSet class
# Registered prefix 'games' creates routes like /api/games/ and /api/games/{id}/
router.register(r'games', GameViewSet)
router.register(r'publishers', PublisherViewSet)

urlpatterns = [
    path('admin/', admin.site.urls),
    # Include all router urls under /api/
    path('api/', include(router.urls)),
]
```

- **Verify**: Start the development server (`python manage.py runserver`), open `http://localhost:8000/api/` in your browser, and verify the DRF browsable API root page displays with links to `/api/games/` and `/api/publishers/`.
- ⚠ **Common Pitfalls**:
  - ○ **`rest_framework` missing from INSTALLED_APPS**: The browsable API page will crash or render without CSS if DRF isn't registered in `settings.py`.
  - ○ **Forgetting `include(router.urls)`**: If you do not include the router's URLs inside `urlpatterns`, the registered routes won't match any requests.

---

4. Check for Understanding (10 min)

> ***Question**: "What does a serializer do with incoming data before it reaches the database?"*
> ***Answer**: A serializer validates the structure and types of incoming data against defined field rules, checks for required fields, runs custom validation logic (e.g., checking if a value is valid), and parses the raw JSON input into a validated Python dictionary (`serializer.validated_data`).*

> ***Question**: "What's the difference between `GET /api/games/` and `GET /api/games/1/`? Which ViewSet methods handle each?"*
> ***Answer**: `GET /api/games/` retrieves a list of all games and is handled by the `list` method of `GameViewSet`. `GET /api/games/1/` retrieves the details of a single game with ID 1 and is handled by the `retrieve` method of `GameViewSet`.*

> ***Question**: "Why might you want `fields = ('id', 'title', 'price')` instead of `'__all__'`?"*
> ***Answer**: Explicitly defining fields improves security and API efficiency by ensuring that sensitive internal fields (like database flags, internal statuses, or relationship keys) are not leaked to the*

> *frontend, and reduces bandwidth by excluding unused fields.*

> ***Question**: "What would happen if we forgot to exclude `webhook_secret` from the publisher serializer?"*
> ***Answer**: The `webhook_secret` would be serialized and returned in public GET/POST responses. This would allow anyone viewing the API output to obtain the secret, enabling them to forge webhook requests and make unauthorized deliveries appear authentic to the publisher's system.*

---

5. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - Navigating to `GET /api/games/` returns `200 OK` with a JSON array.
  - Sending a `POST` request to `/api/games/` with a valid payload (title, publisher, price) successfully creates a record and returns `201 Created`.
  - Navigating to `/api/publishers/` returns the publisher list and does NOT contain `webhook_secret` in the JSON response payload.